

PEB 结构深度解析与内存定位

● 为什么需要手工遍历 PEB

在红队开发与恶意软件编写中，有个基础需求：获取已加载模块（如 kernel32.dll、ntdll.dll）的基址，以便后续解析导出表、动态调用敏感 API。
常规做法是调用 GetModuleHandle 或 LoadLibrary。但这些 API 是用户态 API，很容易被 EDR/杀软通过 API 钩子（Inline Hook/IAT Hook）监控。

只要程序调用了 GetModuleHandle，EDR 会立刻记录：

- 我们访问了哪个 DLL？
- 我们的调用栈是怎样的？
- 我们紧接着用它做了什么？

绕过方法：不再调用这些 API，而是直接从进程的“户口本”——PEB（进程环境块）中读取模块列表。这种操作只涉及 CPU 寄存器和内存读取，**完全不会触发用户态 API 调用**，从而避开多数 API 挂钩监控。

● 核心概念与数据结构

在动手之前，需要了解几个关键结构体。这里只展示与遍历相关的字段，其余忽略。

TEB（线程环境块）与 gs:[0x60]

- 在 x64 Windows 中，gs 段寄存器指向当前线程的 TEB。
- TEB 偏移 0x60 处存放着 PEB 的指针。
- 读取方法：uintptr_t pPeb = __readgsqword(0x60);

PEB（进程环境块）

- PEB 是进程级别的结构体，里面包含了 Ldr 指针。
- 在 x64 下，PEB->Ldr 位于 PEB + 0x18。
- Ldr 指向 PEB_LDR_DATA 结构体，该结构体管理着进程已加载的模块列表。

PEB_LDR_DATA

在 PEB_LDR_DATA 偏移 0x20 处，有 InMemoryOrderModuleList，它是一个 LIST_ENTRY 双向链表的头节点。

LDR_DATA_TABLE_ENTRY

每个已加载的模块对应一个 LDR_DATA_TABLE_ENTRY 结构体。我们关心的字段有：

偏移 (x64)	字段名	说明
0x10	InMemoryOrderLinks	链表节点(LIST_ENTRY)，用于串联所有模块
0x30	DllBase	模块基址(PVOID)
0x58	BaseDllName	UNICODE_STRING，模块文件名（不含路径）

> 注意：这些偏移量在 Windows 10/11 x64 下稳定，但在其他版本或 32 位下会有所不同。

LIST_ENTRY（双向链表节点）

```
typedef struct _LIST_ENTRY {
    struct _LIST_ENTRY *Flink; // 前向指针
    struct _LIST_ENTRY *Blink; // 后向指针
}
```

```
} LIST_ENTRY;
```

UNICODE_STRING (宽字符串结构)

```
typedef struct _UNICODE_STRING {  
    USHORT Length;           // 字符串字节数 (不是字符数)  
    USHORT MaximumLength;    // 缓冲区大小  
    PWSTR Buffer;            // 指向实际宽字符串的指针  
} UNICODE_STRING;
```

● 手工遍历流程

- 通过 `_readgsqword(0x60)` 获取 PEB 指针。
- 从 `PEB + 0x18` 获取 Ldr 指针。
- 从 `Ldr + 0x20` 获取 `InMemoryOrderModuleList` 的头节点。
- 从头节点的 `Flink` 开始，依次遍历每个 `LIST_ENTRY`。
- 对于每个 `LIST_ENTRY`，通过 `CONTAINING_RECORD` 宏（或手动 - 0x10）反推出 `LDR_DATA_TABLE_ENTRY` 的起始地址。
- 读取 `DllBase (+0x30)` 和 `BaseDllName (+0x58)` 获取基址和文件名。
- 继续 `Flink` 直到回到头节点。

● 为什么这很重要（红队/免杀意义）

- 隐蔽性：全程不调用 `GetModuleHandle`、`LoadLibrary` 等 API，避免 EDR 在用户态 API 上的钩子。
- 定位动态 API：拿到 `kernel32.dll` 基址后，可以手工解析其导出表，获取 `GetProcAddress` 和 `LoadLibrary` 的真实地址，进而动态解析任何所需函数，完全绕开 `GetProcAddress` API。
- 低特征：此类代码模式已被现代 EDR 检测，但理解它是学习更高级规避技术（如直接系统调用）的基础。

● 常见问题

- (一) 编译报错“`_readgsqword`”未定义：确保使用 x64 编译模式，该函数是 x64 特有的编译器内部函数。
- (二) 为什么得到的是 0x0 或无效地址？可能是因为 32 位模式下编译，或 Windows 版本偏移量不同。请确认系统是 x64 Windows 10/11。
- (三) 能否同时获取模块大小？可以，读取 `pModule->SizeOfImage`（偏移 0x40）即可。
- (四) 是否可用于其他进程？此方法读取的是自身进程的 PEB。要读取其他进程，需要使用 `NtReadVirtualMemory` 结合 `NtQueryInformationProcess` 获取目标 PEB 地址，属于高级技术。